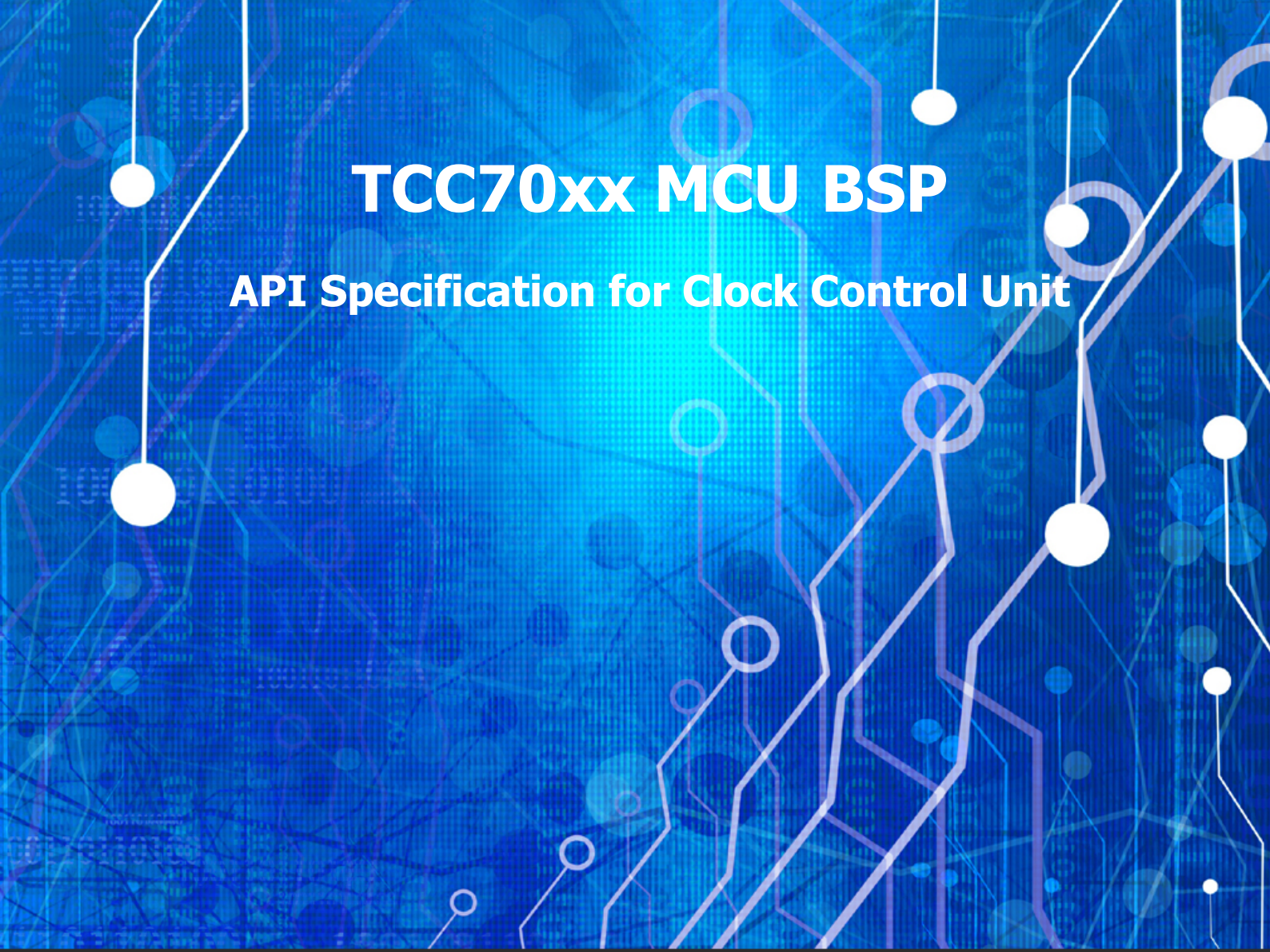




TCC70xx MCU BSP

API Specification for Clock Control Unit

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files	4
4	Enumerations	5
4.1	CLOCKPll_t.....	5
4.2	CLOCKMpll_t	6
4.3	CLOCKPclkctrlSel_t	7
4.4	CLOCKPeri_t.....	8
4.5	CLOCKIobus_t	10
5	Constants.....	12
5.1	Clock Rate	12
6	Structures	13
6.1	CLOCKPms_t.....	13
6.2	CLOCKClkCtrl_t.....	14
6.3	CLOCKPclkCtrl_t	15
7	APIs	16
7.1	CLOCK_Init	16
7.2	CLOCK_SetPllRate.....	17
7.3	CLOCK_GetPllRate	18
7.4	CLOCK_SetPllDiv.....	19
7.5	CLOCK_SetClkCtrlRate.....	20
7.6	CLOCK_GetClkCtrlRate	21
7.7	CLOCK_IsPeriEnabled.....	22
7.8	CLOCK_EnablePeri	23
7.9	CLOCK_DisablePeri	24
7.10	CLOCK_GetPeriRate	25
7.11	CLOCK_SetPeriRate.....	26
7.12	CLOCK_IsIobusPwdn.....	27
7.13	CLOCK_EnableIobus	28
7.14	CLOCK_SetIobusPwdn.....	29
7.15	CLOCK_SetSwReset	30
8	How to use Clock Driver	31
8.1	Initialize Clock Driver	31
8.2	Set Peripheral Clock	31
8.3	Set Peripheral IO Bus Clock	31
9	References	32
10	Revision History	33
	Rev. 0.10: 2021-09-03	33

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the clock control unit device driver (hereinafter referred to as CLOCK driver) of TCC70xx MCU BSP. For more detailed information on clock control unit, refer to Chapter 21 Clock Control Unit in “*TCC70xx Full Specification*” [1].

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

CCU	Clock Control Unit
PLL	Phase-Locked Loop
XIN	Clock Signal from PMU (Crystal Oscillator Clock)
XTIN	Clock Signal from RTC
VCO	Voltage-controlled Oscillator
FIN	Input frequency
FOUT	Output frequency
PCLK	Peripheral Clock

3 SOURCE FILES

3.1 Location of Source Files

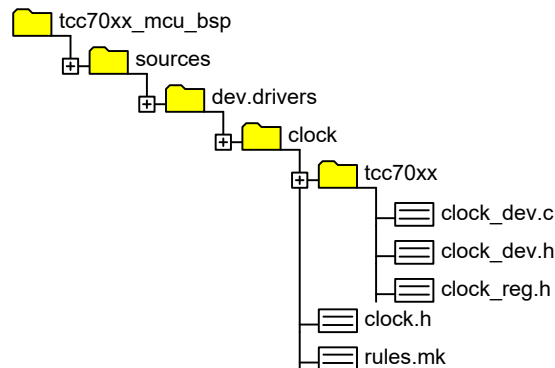


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- clock_dev.c
 - Clock driver source
- clock_dev.h
 - Clock driver header
- clock_reg.h
 - Defines of clock register
- clock.h
 - External APIs header to set clock
- rules.mk
 - Rules used for build

4 ENUMERATIONS

4.1 CLOCKPLL_t

Enumeration variables for PLL channel index

Declaration

```
typedef enum __CLOCK_PLL__  
{  
    CLOCK_PLL_MICOM_0 = 0,  
    CLOCK_PLL_MICOM_1 = 1  
} CLOCKPLL_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>CLOCK_PLL_MICOM_0</i>	<i>PLL0 selection index</i>
<i>CLOCK_PLL_MICOM_1</i>	<i>PLL1 selection index</i>

Remark

None

4.2 CLOCKMpll_t

Enumeration variables for MCU PLL source index

Declaration

```
typedef enum __CLOCK_M_PLL__  
{  
    CLOCK_MPLL_0      = 0,  
    CLOCK_MPLL_1      = 1,  
    CLOCK_MPLL_DIV_0   = 2,  
    CLOCK_MPLL_DIV_1   = 3,  
    CLOCK_MPLL_XIN     = 4  
} CLOCKMpll_t;
```

Description

Value	Description
CLOCK_MPLL_0	PLL0 for FIN source clock
CLOCK_MPLL_1	PLL1 for FIN source clock
CLOCK_MPLL_DIV_0	Divided PLL0 for FIN source clock
CLOCK_MPLL_DIV_1	Divided PLL1 for FIN source clock
CLOCK_MPLL_XIN	XIN for FIN source clock

Remark

None

4.3 CLOCKPclkctrlSel_t

Enumeration variables for MCU Peripheral PLL source index

Declaration

```
typedef enum __CLOCK_PCLK_CTRL_SEL__
{
    CLOCK_PCLKCTRL_SEL_PLL0      = 0,
    CLOCK_PCLKCTRL_SEL_PLL1      = 1,
    CLOCK_PCLKCTRL_SEL_XIN       = 5,
    CLOCK_PCLKCTRL_SEL_PLL0DIV    = 10,
    CLOCK_PCLKCTRL_SEL_PLL1DIV    = 11,
    CLOCK_PCLKCTRL_SEL_XINDIV     = 23
} CLOCKPclkctrlSel_t;
```

Description

Value	Description
CLOCK_PCLKCTRL_SEL_PLL0	PLL0 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL1	PLL1 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_XIN	XIN output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL0DIV	Divided PLL0 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL1DIV	Divided PLL1 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_XINDIV	Divided XIN output for peripheral source clock selection

Remark

None

4.4 CLOCKPeri_t

Enumeration variables for peripheral index

Declaration

```
typedef enum CLOCK_PERI
{ /* Peri. Name */
    // MICOM Peri
    CLOCK_PERI_SFMC      = 0,
    CLOCK_PERI_CAN0      = 1,
    CLOCK_PERI_CAN1      = 2,
    CLOCK_PERI_CAN2      = 3,
    CLOCK_PERI_GPSB0     = 4,
    CLOCK_PERI_GPSB1     = 5,
    CLOCK_PERI_GPSB2     = 6,
    CLOCK_PERI_GPSB3     = 7,
    CLOCK_PERI_GPSB4     = 8,
    CLOCK_PERI_UART0     = 9,
    CLOCK_PERI_UART1     = 10,
    CLOCK_PERI_UART2     = 11,
    CLOCK_PERI_UART3     = 12,
    CLOCK_PERI_UART4     = 13,
    CLOCK_PERI_UART5     = 14,
    CLOCK_PERI_I2C0      = 15,
    CLOCK_PERI_I2C1      = 16,
    CLOCK_PERI_I2C2      = 17,
    CLOCK_PERI_I2C3      = 18,
    CLOCK_PERI_I2C4      = 19,
    CLOCK_PERI_I2C5      = 20,
    CLOCK_PERI_PWM0      = 21,
    CLOCK_PERI_PWM1      = 22,
    CLOCK_PERI_PWM2      = 23,
    CLOCK_PERI_ICTC0     = 24,
    CLOCK_PERI_ICTC1     = 25,
    CLOCK_PERI_ICTC2     = 26,
    CLOCK_PERI_ICTC3     = 27,
    CLOCK_PERI_ICTC4     = 28,
    CLOCK_PERI_ICTC5     = 29,
    CLOCK_PERI_ADC0      = 30,
    CLOCK_PERI_ADC1      = 31,
    CLOCK_PERI_TIMER0    = 32,
    CLOCK_PERI_TIMER1    = 33,
    CLOCK_PERI_TIMER2    = 34,
    CLOCK_PERI_TIMER3    = 35,
    CLOCK_PERI_TIMER4    = 36,
    CLOCK_PERI_TIMER5    = 37,
    CLOCK_PERI_TIMER6    = 38,
    CLOCK_PERI_TIMER7    = 39,
    CLOCK_PERI_TIMER8    = 40,
    CLOCK_PERI_TIMER9    = 41,
    CLOCK_PERI_I2S0      = 42,
    CLOCK_PERI_I2S1      = 43,
    CLOCK_PERI_GMAC0     = 44,
    CLOCK_PERI_GMAC1     = 45,
    CLOCK_PERI_MAX       = 46
} CLOCKPeri_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>CLOCK_PERI_xxxx</i>	<i>Peripheral clock control offset. Refer to Register Map (0xA0F24A0F24000) in CCU in “TCC70xx Full Specification” [1].</i>
<i>CLOCK_PERI_MAX</i>	<i>Maximum Peripheral clock control offset</i>

Remark

None

4.5 CLOCKIobus_t

Enumeration variables for peripheral's IO bus index

Declaration

```
typedef enum CLOCK_IO_BUS
{
    CLOCK_IOBUS_SFMC           = 0,
    CLOCK_IOBUS_IMC            = 1,
    CLOCK_IOBUS_PFLASH         = 2,
    CLOCK_IOBUS_DFLASH         = 3,
    CLOCK_IOBUS_GIC            = 4,
    CLOCK_IOBUS_SOC400         = 5,
    CLOCK_IOBUS_DMA_CON0       = 6,
    CLOCK_IOBUS_DMA_CON1       = 7,
    CLOCK_IOBUS_DMA_CON2       = 8,
    CLOCK_IOBUS_DMA_CON3       = 9,
    CLOCK_IOBUS_DMA_CON4       = 10,
    CLOCK_IOBUS_DMA_CON5       = 11,
    CLOCK_IOBUS_DMA_CON6       = 12,
    CLOCK_IOBUS_DMA_CON7       = 13,
    CLOCK_IOBUS_CAN0           = 14,
    CLOCK_IOBUS_CAN1           = 15,
    CLOCK_IOBUS_CAN2           = 16,
    CLOCK_IOBUS_CAN_CONF       = 17,
    CLOCK_IOBUS_UART0          = 18,
    CLOCK_IOBUS_UART1          = 19,
    CLOCK_IOBUS_UART2          = 20,
    CLOCK_IOBUS_UART3          = 21,
    CLOCK_IOBUS_UART4          = 22,
    CLOCK_IOBUS_UART5          = 23,
    CLOCK_IOBUS_CONF           = 24,
    CLOCK_IOBUS_I2C0           = 25,
    CLOCK_IOBUS_I2C1           = 26,
    CLOCK_IOBUS_I2C2           = 27,
    CLOCK_IOBUS_I2C3           = 28,
    CLOCK_IOBUS_I2C4           = 29,
    CLOCK_IOBUS_I2C5           = 30,
    CLOCK_IOBUS_I2C_M_PORTCFG   = 31,
    CLOCK_IOBUS_PWM0           = 32, /* HCLK_MASK1[0] */
    CLOCK_IOBUS_PWM1           = 33, /* HCLK_MASK1[1] */
    CLOCK_IOBUS_PWM2           = 34, /* HCLK_MASK1[2] */
    CLOCK_IOBUS_PWM_CONF       = 35, /* HCLK_MASK1[3] */
    CLOCK_IOBUS_ICTC0          = 36, /* HCLK_MASK1[4] */
    CLOCK_IOBUS_ICTC1          = 37, /* HCLK_MASK1[5] */
    CLOCK_IOBUS_ICTC2          = 38, /* HCLK_MASK1[6] */
    CLOCK_IOBUS_ICTC3          = 39, /* HCLK_MASK1[7] */
    CLOCK_IOBUS_ICTC4          = 40, /* HCLK_MASK1[8] */
    CLOCK_IOBUS_ICTC5          = 41, /* HCLK_MASK1[9] */
    CLOCK_IOBUS_ADC0           = 42, /* HCLK_MASK1[10] */
    CLOCK_IOBUS_ADC1           = 43, /* HCLK_MASK1[11] */
    CLOCK_IOBUS_TIMER0         = 44, /* HCLK_MASK1[12] */
    CLOCK_IOBUS_TIMER1         = 45, /* HCLK_MASK1[13] */
    CLOCK_IOBUS_TIMER2         = 46, /* HCLK_MASK1[14] */
    CLOCK_IOBUS_TIMER3         = 47, /* HCLK_MASK1[15] */
    CLOCK_IOBUS_TIMER4         = 48, /* HCLK_MASK1[16] */
}
```

```

CLOCK_IOPBUS_TIMER5      = 49, /* HCLK_MASK1[17] */
CLOCK_IOPBUS_TIMER6      = 50, /* HCLK_MASK1[18] */
CLOCK_IOPBUS_TIMER7      = 51, /* HCLK_MASK1[19] */
CLOCK_IOPBUS_TIMER8      = 52, /* HCLK_MASK1[20] */
CLOCK_IOPBUS_TIMER9      = 53, /* HCLK_MASK1[21] */
CLOCK_IOPBUS_GPSB0       = 54, /* HCLK_MASK1[22] */
CLOCK_IOPBUS_GPSB1       = 55, /* HCLK_MASK1[23] */
CLOCK_IOPBUS_GPSB2       = 56, /* HCLK_MASK1[24] */
CLOCK_IOPBUS_GPSB3       = 57, /* HCLK_MASK1[25] */
CLOCK_IOPBUS_GPSB4       = 58, /* HCLK_MASK1[26] */
CLOCK_IOPBUS_GPSB_CONF   = 59, /* HCLK_MASK1[27] */
CLOCK_IOPBUS_GPSB_SM     = 60, /* HCLK_MASK1[28] */
CLOCK_IOPBUS_I2S         = 61, /* HCLK_MASK1[29] */
CLOCK_IOPBUS_GMAC        = 62, /* HCLK_MASK1[30] */
CLOCK_IOPBUS_RESERVED    = 63, /* HCLK_MASK1[31] */
CLOCK_IOPBUS_WDT         = 64, /* HCLK_MASK2[0] */
CLOCK_IOPBUS_GPIO        = 65, /* HCLK_MASK2[1] */
CLOCK_IOPBUS_CMU         = 66, /* HCLK_MASK2[2] */
CLOCK_IOPBUS_SYSSM       = 67, /* HCLK_MASK2[3] */
CLOCK_IOPBUS_MAX         = 68
} CLOCKIobus_t;

```

Description

Value	Description
CLOCK_IOPBUS_XXXX	Peripheral's IO bus clock control offset. Refer to System Configuration Register Map (0xA0F20000) in "TCC70xx Full Specification" [1].
CLOCK_IOPBUS_MAX	Maximum Peripheral clock control offset

Remark

None

5 CONSTANTS

5.1 Clock Rate

Constant value for the pre-defined reference clock rate

Declaration

```
#define CLOCK_XIN_CLK_RATE    (12000000UL)    // 12MHz
```

Description

<i>Value</i>	<i>Description</i>
<i>CLOCK_XIN_CLK_RATE</i>	<i>Clock rate 12 MHz</i>

Remark

None

6 STRUCTURES

6.1 CLOCKPms_t

Structure for PLL rate control

Declaration

```
typedef struct CLOCK_PMS
{
    uint32 uiFpLL;
    uint32 uiEn;
    uint32 uiP;
    uint32 uiM;
    uint32 uiS;
    uint32 uiSrc;
} CLOCKPms_t;
```

Description

Value	Description
uiFpLL	PLL value
uiEn	RESETB: PLL Reset when Active Low This signal is used for power-down control 0: Power-down mode is enabled and all digital blocks are reset. 1: PLL starts its normal operation after lock time. (Refer to 0xA0F24000 in "TCC70xx Full Specification" [1])
uiP	P: Value of the 6-bit programmable pre-divider. (Refer to 0xA0F24000 in "TCC70xx Full Specification" [1])
uiM	M: Value of the 10-bit programmable main-divider. (Refer to 0xA0F24000 in "TCC70xx Full Specification" [1])
uiS	S: Value of the 3-bit programmable Scaler. (Refer to 0xA0F24000 in "TCC70xx Full Specification" [1])
uiSrc	FIN_SEL: FIN source clock selection. (Refer to 0xA0F24000 in "TCC70xx Full Specification" [1])

Remark

None

6.2 CLOCKClkCtrl_t

Structure for PLL clock control register

Declaration

```
typedef struct CLOCK_CLK_CTRL
{
    uint32 uiFreq;
    uint32 uiEn;
    uint32 uiConf;
    uint32 uiSel;
} CLOCKClkCtrl_t;
```

Description

Value	Description
uiFreq	Clock rate
uiEn	Enable
uiConf	XIN divider's divisor
uiSel	Source clock selection

Remark

None

6.3 CLOCKPclkCtrl_t

Structure for peripheral PLL clock control register

Declaration

```
typedef struct CLOCK_PCLK_CTRL
{
    uint32 uiPeriName;
    uint32 uiFreq;
    uint32 uiMd;
    uint32 uiEn;
    uint32 uiSel;
    uint32 uiDivVal;
} CLOCKPclkCtrl_t;
```

Description

Value	Description
uiPeriName	Peripheral name
uiFreq	Clock rate
uiMd	Mode divider
uiEn	Enable
uiSel	Source clock
uiDivVal	Divisor value

Remark

None

7 APIs

7.1 CLOCK_Init

Initializes the clock controller.

Declaration

```
void CLOCK_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

7.2 CLOCK_SetPIIRate

Controls the *PLL0PMS* and *PLL1PMS* register. (Refer to *0xA0F24000* in “*TCC70xx Full Specification*” [1].)

Declaration

```
sint32 CLOCK_SetPLLRate
(
    sint32 iId,
    uint32 uiRate
);
```

Parameters

Value	Description
iId	[in] MPLL index
uiRate	[in] Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.3 CLOCK_GetPllRate

Gets the PLL rate.

Declaration

```
uint32 CLOCK_GetPllRate
(
    sint32 iId
);
```

Parameters

Value	Description
iId	[in] Clock index

Return

Value	Description
uint32	Clock rate

Remark

None

7.4 CLOCK_SetPLLDiv

Sets the PLL divisor value.

Declaration

```
sint32 CLOCK_SetPLLDiv  
(  
    sint32 iId,  
    uint32 uiPLLDiv  
);
```

Parameters

Value	Description
<i>iId</i>	<i>[in] Clock index</i>
<i>uiPLLDiv</i>	<i>[in] PLL divisor value</i>

Return

Value	Description
<i>sint32</i>	<i>0: Success -1: Failure</i>

Remark

None

7.5 CLOCK_SetClkCtrlRate

Controls the *CPU_CLK_CTRL* register.

Declaration

```
sint32 CLOCK_SetClkCtrlRate
(
    sint32 iId,
    uint32 uiRate
);
```

Parameters

Value	Description
iId	[in] Clock index
uiRate	[in] Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.6 CLOCK_GetClkCtrlRate

Gets the clock rate for *CPU_CLK_CTRL* register.

Declaration

```
uint32 CLOCK_GetClkCtrlRate
(
    sint32 iId
);
```

Parameters

Value	Description
iId	[in] Clock index

Return

Value	Description
uint32	Clock rate

Remark

None

7.7 CLOCK_IsPeriEnabled

Checks the peripheral enable status.

Declaration

```
sint32 CLOCK_IsPeriEnabled
(
    sint32 iId
);
```

Parameters

Value	Description
iId	[in] Clock index

Return

Value	Description
sint32	Clock rate

Remark

None

7.8 CLOCK_EnablePeri

Enables the peripheral clock.

Declaration

```
sint32 CLOCK_EnablePeri
(
    sint32 iId
);
```

Parameters

Value	Description
iId	[in] Clock index

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.9 CLOCK_DisablePeri

Disables the peripheral clock.

Declaration

```
sint32 CLOCK_DisablePeri  
(  
    sint32 iId  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>iId</i>	<i>[in]</i> Clock index

Return

<i>Value</i>	<i>Description</i>
<i>sint32</i>	0: Success -1: Failure

Remark

None

7.10 CLOCK_GetPeriRate

Gets the peripheral clock rate.

Declaration

```
uint32 CLOCK_GetPeriRate
(
    sint32 iId
);
```

Parameters

Value	Description
iId	[in] Clock index

Return

Value	Description
uint32	0: Success -1: Failure

Remark

None

7.11 CLOCK_SetPeriRate

Sets the peripheral clock rate (*CLOCK_MCKC_PCLKCTRL* register).

Declaration

```
sint32 CLOCK_SetPeriRate
(
    sint32 iId,
    uint32 uiRate
);
```

Parameters

Value	Description
iId	[in] Clock index
uiRate	[in] Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.12 CLOCK_IsIobusPwn

Gets the peripheral's IO bus clock power-down mask status (*HCLK_MASK#* register).

Declaration

```
sint32 CLOCK_IsIobusPwn
(
    sint32 iId,
);
```

Parameters

Value	Description
iId	[in] Bus clock index

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.13 CLOCK_EnableIobus

Enables peripheral's IO bus clock power-down (*HCLK_MASK#*, *SW_RESET#* register).

Declaration

```
sint32 CLOCK_EnableIobus
(
    sint32    iId,
    boolean   bEn
);
```

Parameters

Value	Description
iId	[in] Bus clock index
bEn	[in] Bus clock enable

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.14 CLOCK_SetIobusPwn

Sets the peripheral's IO bus clock power-down mask status (*HCLK_MASK#* register).

Declaration

```
sint32 CLOCK_SetIobusPwn
(
    sint32    iId,
    boolean    bPwn
);
```

Parameters

Value	Description
iId	[in] Bus clock index
bPwn	[in] Bus clock power-down enable

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.15 CLOCK_SetSwReset

Controls the *SW_RESET#* register.

Declaration

```
sint32 CLOCK_SetSwReset
(
    sint32    iId,
    boolean   bReset
);
```

Parameters

Value	Description
iId	[in] Bus clock index
bReset	[in] Bus clock reset enable

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

8 HOW TO USE CLOCK DRIVER

The clock driver must be initialized first in the BSP initialization process. Each driver that requires a clock must set the peripheral clock during its own initialization.

8.1 Initialize Clock Driver

```
{  
    CLOCK_Init();  
    ...  
}
```

8.2 Set Peripheral Clock

```
{  
    ...  
    uint32 rate = 48000000UL;  
  
    /* Enable/Disable the xxx controller peripheral clock */  
    regRet = CLOCK_SetPeriRate(CLOCK_PERI_xxx, rate);  
    regRet = CLOCK_EnablePeri(CLOCK_PERI_xxx);  
    rate = CLOCK_GetPeriRate(CLOCK_PERI_xxx);  
    printf("rate is %x \n", rate);  
    ...  
    regRet = CLOCK_DisablePeri(CLOCK_PERI_xxx);  
    ...  
}
```

8.3 Set Peripheral IO Bus Clock

```
{  
    ...  
    /* Enable/Disable the xxx controller peripheral bus clock*/  
    regRet = CLOCK_EnableIobus(CLOCK_IOBUS_xxx, SALEnabled);  
    regRet = CLOCK_EnableIobus(CLOCK_IOBUS_xxx, SALDisabled);  
}
```

9 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."